

Malith - Bilingual Security Video Guide

Sinhala action guide + English speech + code segments + test evidence

Member	R.M.M.P. Bandara (Malith)
Original website	http://localhost:5174/login
Secure website	http://localhost:5173/login
Assigned scope	V-04, V-10 and V-11 - business integrity, pricing authority and auditability

Common Sinhala guide

මුලින් මේ පොදු setup එක බලලා browser එක සූදානම් කරන්න

Demo setup සහ record කරන පොදු ක්‍රියා

- Original website: http://localhost:5174/login.
- Secure website: http://localhost:5173/login.
- Username: cashier.colombo. Local demo password එක login field එකට type කරන්න; password එක video එකේ නොපෙන්වන්න.
- Browser tabs දෙකක් තියාගෙන Ctrl + Tab භාවිතා කර Original සහ Secure tabs අතර මාරු වන්න.
- හැම action එකකටම cursor එක field/button/result එක උඩට ගෙන තත්පර දෙකක් නවත්වලා screenshot ගන්න.
- මුලින් Original result එක පෙන්වන්න; පස්සේ ඒ bug එක කියන්න; ඊළඟට Secure result, code segment සහ test result පෙන්වන්න.
- Live database එකට අනවශ්‍ය destructive action කරන්න එපා. Test database, code සහ existing evidence භාවිතා කරන්න.

Video introduction එකේ කියන්න

මමෙ video එකේදී මම Pubudu Tire Management System එකේ මට assign කරලා තිබෙන security workstream එක පෙන්වනවා. හැම vulnerability එකකටම මුලින් Original version එකේ අවදානම පෙන්වනවා. ඊට පස්සේ Secure version එකේ කළ fix එක, අදාළ code segment එක සහ automated test evidence එක පෙන්වනවා. Password, token, secret හෝ personal account information screen එකේ පෙන්වන්නේ නැහැ.

English introduction - say this

"Hello. In this section I will demonstrate my assigned security workstream in the Pubudu Tire Management System. For each vulnerability, I will first show the Original behavior, explain the security impact, and then show the Secure behavior, the relevant code change, and the automated test evidence. Passwords, tokens, secrets and personal account information are kept hidden."

V-04 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කතාට ස කරන විදිහ

Video steps

Original tab එකේ customer update flow හමේ original CustomerController.php පෙන්වන්න.
credit_balance වගේ protected field එකක් request එකට දාන තැන පෙන්වන්න. Secure tab/code එකේ prohibited rules, \$validated update සහ 422 test එක පෙන්වන්න. Screenshot names:
V04-01-original-mass-assignment.png, V04-02-secure-validation.png,
V04-03-business-integrity-test.png.

V-04 - mass assignment හා customer debt

කලින් මොකද වුණේ?

CustomerController::update() request එක validate කළත් පසුව \$request->all() model update එකට යැව්වා. Laravel validate() method එක "මමෙ data හරිද?" ලෙස එක; payload එකෙන් අනෙක් keys සියල්ල ඉවත් කරන filter එකක් තොටවේ. \$fillable list එකේ වැරදි field තිබුණොත් request එකේ ඒ field ද model එකට යනවා. Attackerට credit_balance: 0 යවා customer debt මකා දැමිය හැකි වුණා; customer_code වැනි business key එකක් වනෙස් කිරීමටත් උත්සාහ කළ හැකි වුණා. Debt වනෙස් වුණත් ledger row එකක් නොතිබීම මුළු වාර්තාවටත් හානියක්.

Fix එකේ flow

```
PUT /api/customers/{id}
→ validation rules
→ credit_balance ■■ customer_code update ■■■■■ prohibited
→ $validated fields ■■■■■ Customer::update ■■■
→ debt ■■■■■ ■■■■■ ■■■■■ credit endpoint + ledger entry
```

app/Http/Controllers/Api/CustomerController.php හි update() තුළ credit_balance/customer_code prohibited, \$customer->update(\$validated) යොදා ඇත.
app/Models/Customer.php හි customer_code mass-assignable \$fillable තුළ නැත. ඒ කියන්නේ controller කනෙකු නැවත \$request->all() භාවිතා කළත් business key එකට දවෙනි ආරක්ෂක තට්ටුවක් ඇත. Customer සිදීමේදී code එක generate කර direct assign කරයි; පසුව වනෙස් නොකළ යුතුයි. Ledger endpoint එක CreditTransaction row තබයි.

Code segment එක තේරුම් ගන්න:

```
$validated = $request->validate([
    'credit_balance' => 'prohibited',
    'customer_code' => 'prohibited',
    // ■■■■■ ■■■■■ ■■■■■ ■■■■■ customer fields ...
]);
$customer->update($validated);
```

මෙය අධීයනය සඳහා සරල කළ fragment එකකි; නිවැරදි සම්පූර්ණ rules CustomerController.php හි ලෙන්න. prohibited යනු silent ignore කිරීමකට වඩා හොඳයි: client එකට 422 සහ හේතුව ලැබෙයි. BusinessIntegrityTest.php හි debt/key update block වීමත් ledger entry ලියවීමත් පරීක්ෂා වේ.

English speech - read this in the video

V-04 - Mass assignment and customer debt

[Show the Original customer update flow or the original controller code.]

"In the Original API, the request was validated, but the controller then passed the complete request payload to the model with `request->all()`. Validation checks whether selected fields are valid; it does not automatically remove every other field. Therefore, an attacker could add a protected field such as `credit_balance` or `customer_code` and change business data without the correct workflow. A debt change could also happen without a corresponding ledger entry."

[Show the Secure controller and the negative test.]

"In the Secure version, protected business fields are prohibited and only the validated data is sent to the model. Credit changes use a dedicated endpoint that creates a ledger transaction. This makes the update explicit, auditable and rejectable with HTTP 422 when a forbidden field is supplied."

Key code segments and file locations

Validated and prohibited business fields

File: demo/secure/pubudu-pos-api-secure/app/Http/Controllers/Api/CustomerController.php Lines: 189-216

```
$validated = $request->validate([
    'credit_balance' => 'prohibited',
    'customer_code' => 'prohibited',
]);
$customer->update($validated);
```

Business key is not mass assignable

File: demo/secure/pubudu-pos-api-secure/app/Models/Customer.php Lines: 18-36

```
protected $fillable = [
    'full_name', 'phone_no', 'email',
    // customer_code is deliberately absent
];
```

V-10 - Sinhala guide first, English speech underneath

Sinhala guide - මේ කොටස කරන විදිහ

Video steps

Original invoice screen/request එක පතේවා item selling price එක browser request එකෙන් වනෙස් කළ හැකි බව කියන්න. Live production data වනෙස් නොකර evidence/code භාවිතා කරන්න. Secure InvoiceController.php, invoicePayload.js සහ price/discount tests පතේවන්න. Screenshot names: V10-01-original-price-tamper.png, V10-02-secure-price-rejected.png, V10-03-server-price-code.png, V10-04-discount-single-use-test.png.

V-10 - cashier browser එකෙන් invoice මිල හදන ගැටලුව

වැරදි assumption එක

Original backend එක totals server එකේ ගණනය කළා. එහෙත් per-item selling_price සහ discount request එකෙන් ගන්නා. ඒ නිසා catalogue මිල උ 32,000 වූ ටයරයක selling_price: 1 යවා serverට "නිවැරදිව" උ 1 total එකක් ගණනය කර save කිරීමට හැකි වුණා. Discount approval screen එක නිවුණත් එය bypass කළ හැකි වුණා. මෙය business logic / trust boundary defect එකකි.

Fixed request-to-database flow

```
Cashier invoice request
→ fields ■■ quantity validate ■■■■■
→ DB transaction ■■■■ ■■■■■ invoice number/stock records lock ■■■■■
→ FIFO stock batches ■■■■ server-side unit price ■■■■■■■■■
→ client ■■■■ price ■■ expected quote ■■■■ compare ■■■■■
→ mismatch ■■■ 422; invoice/stock change ■■■■
→ discount ■■■■ ■■■ ■■■ cashier ■■■■ approved + unused request ■■■■■
→ approval amount ■■■■■■■■ ■■■ 422
→ approval single-use ■■■■ consumed_at/consumed_inv_no set ■■■■■
→ server-derived total, customer credit, stock movement ■■ invoice save
```

app/Http/Controllers/Api/InvoiceController.php හි store(), resolve...Price helper, resolveApprovedDiscount() හා stock allocation code බලන්න. Stock batches කිහිපයකින් item quantity ලබා ගත් විට ඒ batch වල මිල quantity-weighted ලෙස ගණනය කරයි. Service line එකක මිල services.price වෙනිත්. lockForUpdate() concurrent sales/approvals අතර එකම stock හෝ discount දවෙරක් භාවිතා වීමේ අවදානම අඩු කරයි. Database transaction එකක් නිසා එක පියවරක් fail වුණොත් sale එකේ අඩක් පමණක් save නොවෙයි.

වැදගත් logic fragment (සංකල්පය):

```
$submitted = (float) $item['selling_price'];
$authoritative = $this->resolvePriceFromStockOrService(...);
if (round($submitted, 2) !== round($authoritative, 2)) {
    throw ValidationException::withMessages([
        "items.$index.selling_price" => 'Price differs from the catalogue.',
    ]);
}
```

මේක exact copy/paste implementation එක නොව flow එක මතක තබා ගැනීමට ඇති pseudocode එකකි. Source එකේ FIFO allocation හා floating point tolerance නව විස්තරාත්මකයි. Client price එක authoritative නොවෙයි; mismatch එක silent correct නොකර reject කරන්නේ cashier screen එකේ customerට පෙන්වූ මිල සහ books වල මිල වෙනස් නොවීමයි.

Frontend එකේ කුමක් වෙනස් කළාද?

src/lib/invoicePayload.js හි buildInvoicePayload() operator තේරූ දේ පමණක් request එකට ගනී. total_amount, net_total, balance, credit_paid, inv_by වැනි server-calculated fields යවන්නේ නැහැ. selling_price නව යවයි, නමුත් expected quote ලෙස පමණයි; backend එක compare කරයි. src/pages/Invoice/CreateInvoice.jsx හි invoice creation paths මෙය භාවිතා කරයි. src/lib/invoicePayload.test.js හි tests 4ක් normal input, server-owned fields නොයැවීම සහ customer/price cases බලනවා.

Theory: Frontend validation UX සඳහා හොඳයි. Security enforcement API එකේ යුතුයි, මන්ද userට browser request එක ඕනෑම ලෙස වෙනස් කළ හැක.

V-10 tests වල කියවන ජූර්නිඑල

tests/Feature/Security/BusinessIntegrityTest.php:

- Tampered unit price 422; stock නොකරෙයි.
- Catalogue price successful invoice.

- Approval නැති discount 422.
- Approved discount sale accepted, approval consumed.
- එක approval එක නැවත භාවිතය / වෙන cashierගේ approval / cap ඉක්මවීම reject.
- Invoice creator (inv_by) token එකතේ; request එකතේ තොවේ.

English speech - read this in the video

V-10 - Client-controlled invoice prices and discount approval

[Show the Original invoice request or the original invoice price-handling code.]

"The Original backend calculated the invoice total on the server, but it trusted the item price and discount supplied by the browser. A cashier could change the price of an expensive tyre to one rupee, and the server would calculate that manipulated value correctly. The discount approval workflow was therefore decorative because the request could bypass it."

[Show the Secure invoice controller and the fixed request payload builder.]

"The Secure backend resolves the authoritative price from stock or service data. The submitted price is treated as an expected quote and is rejected if it does not match. A discount must belong to the correct cashier, be approved, stay within its limit and be consumed only once. Server-derived totals, credit balances and invoice ownership are not trusted from the client."

Key code segments and file locations

Server-authoritative price check

File: demo/secure/pubudu-pos-api-secure/app/Http/Controllers/Api/InvoiceController.php Lines: 64-134 and 615-696

```
$submitted = (float) $item['selling_price'];
$authoritative = $this->resolveUnitSellingPrice($item);
if (round($submitted, 2) !== round($authoritative, 2)) {
    throw ValidationException::withMessages([
        'selling_price' => 'Price differs from the catalogue.',
    ]);
}
```

Client sends only the minimum invoice payload

File: demo/secure/pubudu-pos-front-end-secure/src/lib/invoicePayload.js Lines: 1-24 and 31-64

```
return {
    inv_date, customer_type, payment_status, items,
    // totals, balance and credit allocations are server-owned
};
```

V-11 - Sinhala guide first, English speech underneath

Sinhala guide - මේ කොටස කරන විදිහ

Video steps

Original invoice cancel flow සහ audit evidence නැති result එක පෙන්වන්න. Secure AuditLogger.php, append-only AuditLog.php, cancellation snapshot සහ audit test එක පෙන්වන්න. Existing demo invoice එකක් destructive ලෙස cancel නොකර dedicated test/evidence භාවිතා කරන්න. Screenshot names:

V11-01-original-cancel-gap.png, V11-02-secure-audit-log.png, V11-03-audit-test.png.

V-11 - audit trail නැති වීම

ඇයි log එකක් අවශ්‍ය?

Cashier වැරදි මිලකට invoice එකක් කර හෝ admin invoice cancel කළොත් “කවුද, කවදාද, කොතැනින්ද, කලින් අගය කීයද?” කියා පසුව දැනගත යුතුයි. Original code එකේ failed logins සහ මුදල් සම්බන්ධ actions ප්රමාණවත් ලෙස record නොවුණා. Invoice cancel කළ විට පැරණි amounts zero වන නිසා pre-cancel value නැති වෙන්නේ පුළුවන්.

Implement කළ පාලනය

database/migrations/...create_audit_logs_table.php මගින් action, severity, actor, subject, old/new values, IP, timestamp වැනි fields ඇති table එකක්. app/Support/AuditLogger.php එකම entry point: event එක database audit_logs table එකටත් වෙනම security log channel එකටත් ලියයි. Sensitive fields recursively redact කරනවා. Logging failure එකක් වුණොත් business action එක නවත්වන්නේ නැහැ; error වෙනම log කරයි. එය availability vs audit durability tradeoff එකකි; viva එකේ කියන්න.

app/Models/AuditLog.php update/delete block කරන append-only model logic එකක්.
app/Http/Controllers/Api/AuditLogController.php හි read API එක admin-only.
InvoiceController.php cancel ක්රියාවට පෙර amount snapshot ගෙන critical event එකක් ලියයි.
CustomerController.php credit adjustment එකට old/new balance සහ reason ලියයි.

Theory: Audit log එක “bug එක නවත්වන” control එකක් නොවේ; incident එක detect/investigate කිරීමට evidence. Database සහ file copy දකෙක් තැබීම recovery ට උපකාරී. එහෙත් absolute tamper-proof කියා නොකියන්න; database/server adminට logs වෙන access තිබිය හැක.

English speech - read this in the video

V-11 - Missing and tamperable audit trail

[Show the Original invoice cancellation behavior or the original cancellation code.]

“In the Original implementation, cancelling an invoice could overwrite its financial values with zero, and there was no reliable record of who performed the action or what the previous values were. This makes a large cancelled sale look similar to a small one and makes financial investigation difficult.”

[Show the Secure audit table, AuditLogger and the cancellation test.]

“The Secure version records security- and money-relevant actions with the actor, subject, old values, new values, IP address and timestamp. The pre-cancellation financial state is captured before values are changed. The audit model is append-only, and the read API is administrator-only. This control mainly supports detection and forensic investigation; authorization and business rules still prevent the unauthorized action.”

Key code segments and file locations

Record the pre-cancellation financial state

File: demo/secure/pubudu-pos-api-secure/app/Http/Controllers/Api/InvoiceController.php Lines: 517-595

```
$before = $invoice->only([
    'total_amount', 'net_total', 'paid_amount'
]);
AuditLogger::record('invoice.cancelled', [
    'old' => $before, 'subject' => ['invoice', $invoice->id],
]);
```

Append-only audit model

File: demo/secure/pubudu-pos-api-secure/app/Models/AuditLog.php Lines: 48-65

```
protected static function booted(): void
{
    static::updating(fn () => throw new LogicException('Audit logs are append-only.'));
    static::deleting(fn () => throw new LogicException('Audit logs are append-only.'));
}
```

Conclusion

Sinhala conclusion

මම memberගේ assigned work එකේදී Original bug එක, Secure fix එක, code evidence එක සහ test result එක පැහැදිලිව පෙන්වන්න. තමන් දායකත්වය කළ contribution එකට පමණක් claim කරන්න. Team-level test counts තමන්ගේ individual count ලෙස කියන්න එපා.

English conclusion - say this

"To conclude, I compared the Original behavior with the Secure behavior, explained the impact of each finding, and connected each fix to source code and automated test evidence. The controls are enforced at the correct boundary, and the remaining limitations are documented rather than hidden. Thank you."

Recording safety

- Passwords, bearer tokens, client secrets and personal Google accounts must stay hidden.
- Use code and test evidence for destructive or unsafe attack steps.
- Keep the cursor on the exact result or code line being discussed.